

System Design Document

IoT Data Lab

M. Rats, V. Gülmez, G. Gharib, C. Stijlaart

Table of Contents

1. Document history	1
2. Introduction	2
3. System Description	3
3.1. Extensions	3
4. System Design	4
4.1. Components	4
4.2. Use Cases	5
4.3. Requirements	6
4.4. Requirements traceability matrix	8
5. System Behavior	9
5.1. Sequence Diagrams	9
5.2. Class Diagram	14
6. Hardware Diagram	15

1. Document history

Table 1. Document History

Version	Date	Status	Author	Description	Remarks
0.1	10-10-2023	Draft	Team	Introduction, Description Component diagram, Hardware Diagram	-
0.2	11-11-2023	Draft	Team	Completed design	-
0.3	29-11-2023	Dradt	Team	Completed system behavior	-

2. Introduction

This document functions as a detailed guide for the design and architecture of the embedded system tasked with capturing, storing and displaying data within a data lab setup. Within its pages, readers will find an exhaustive system description, accompanied by a various diagrams, which define the system architecture and behaviour.

3. System Description

The system is designed to capture, store and showcase various data within a lab setup. The system aims to allow for the user to make use of IoT lab setup on a smaller scale. Besides that, the IoT lab setup aims to simplify the extend upon the system, ensuring various types of experiments can be run.

3.1. Extensions

Should

- The cameras should monitor specific aspects of the test setup, including:
 - Detecting the presence of liquid in the vial.
 - Determining the color of the liquid.
 - Confirming the insertion of a pipette into the vial.
- The prototype should be designed as a basic and straightforward test setup.
- Consideration should be given to what type of data should be stored (e.g., complete footage, partial footage, or data within the footage).

Could

- The prototype could have the potential for extension or upgrade.
- Optimal camera angles for data collection could be identified.
- The system could have a user interface
- Options to extend the system could be explored, such as:
 - Adding multiple camera's
 - Adding different types of cameras (e.g., infrared camera).
 - Integrating extra sensors to obtain additional and diverse data.

4. System Design

4.1. Components

In an embedded system, there are several key parts that work together to make everything function smoothly. These components each have their own specific jobs, but they all rely on each other to make the system work effectively. In this section, we'll take a closer look at these important parts to understand what they do and why they matter in the project.

Table 2. Component Overview

Name	Description
Controller Unit	The controller unit serves as the central processing unit for the system.
Database	The database stores and manages data generated by the system.
Sensor Unit	The sensor unit is the control point for all attached sensors.
Camera Unit	The camera unit is the control point and processor for the attached camera(s).
Dashboard	The interface between application and real-world.

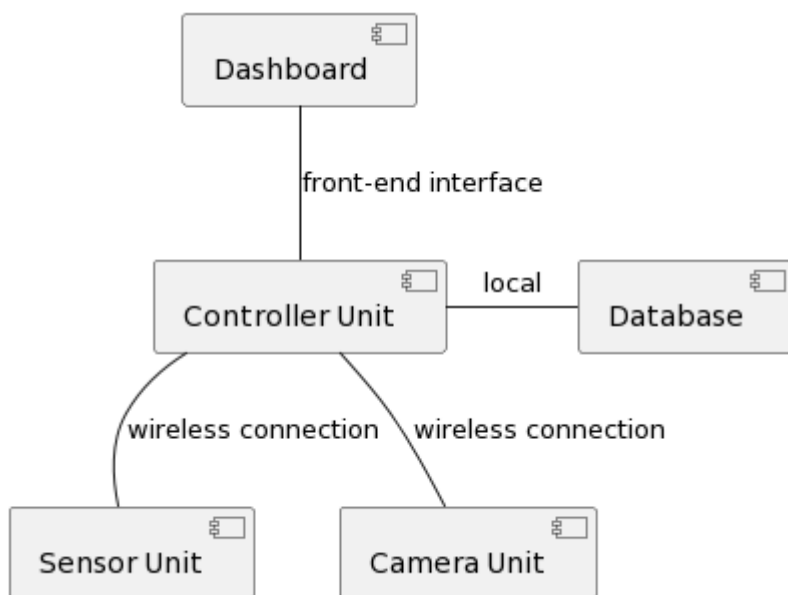


Figure 1. Component Diagram

4.2. Use Cases

Use Case ID : UC_001	Use case : Measuring data from a lab setup
Description	When a chemical process is being executed in a lab set-up, the data is measured through the camera- and sensor unit, and communicated to the controller unit.
Actor	Camera Unit, Sensor Unit, Controller Unit
Use Case ID : UC_002	Use case : Measured data stored in database
Description	When measured data is retrieved on the controller unit, the data is sent to the database to be stored.
Actor	Database, Controller Unit
Use Case ID : UC_003	Use case : Retrieving data from database
Description	When the controller unit receives a request from the front-end application, the controller unit requests the database for the data and communicates this back to the front-end application.
Actor	Database, Controller Unit, Front-end Application

4.3. Requirements

4.3.1. User Requirements

User Requirement ID	Description	Use Cases
UR_001	The user should be able to retrieve the data, that is designated for them.	UC_003
UR_002	The user should be able to make use of the measurement setup in a simple manner	UC_001/UC_002
UR_003	The user should be able to start and stop measuring when making use of the measurement setup.	UC_001
UR_004	The user should be able to stream the current experiment, that takes place in the measurement setup.	UC_001
UR_005	The should get an indication when a change has occurred within the measurement setup.	UC_001

4.3.2. Functional Requirements

Functional Requirement ID	Description	UC/UR
FR_001	The camera unit must make use of one or more cameras to monitors the lab setup	UR_003/UR_004
FR_002	The camera unit must detect presence of liquid in a vial, in the current experiment	UR_003/UR_005
FR_002	The camera unit must identify the color of liquid in a vial, in the current experiment	UR_003/UR_005
FR_003	The controller unit and database safeguards the data, to ensure it is only accessible to the specified group(s).	UC_002/UC_003
FR_004	The database must make use of encrypted data storing in the database.	UR_001

Functional Requirement ID	Description	UC/UR
FR_005	The controller unit and database must only allow the user(s), who 'created' the data, to get access to the data unless specified otherwise.	UR_001
FR_006	All measured data, from the camera unit and sensor unit, is stored in the database.	UR_003
FR_007	The dashboard must show the retrieved data	UR_001
FR_008	Extra sensors must be able to be added to the system	UR_002
FR_009	The controller unit must start and stop measuring based upon the dashboard input	UR_003
FR_010	The controller unit must notify the dashboard when a change in the measurement setup has occurred	UR_005

4.3.3. Non-functional Requirements

Non-Functional Requirement ID	Description	UC/UR
NR_001	The dashboard, with the data, is easily readable	UC_003/UR_001
NR_002	A secure (wireless) connection is established.	-

4.4. Requirements traceability matrix

Table 3. Use cases vs user requirements coverage

	UC_001	UC_002	UC_003
UR_001			X
UR_002	X	X	
UR_003	X		
UR_004	X		
UR_005	X		

Table 4. Functional requirements vs user requirements coverage

	UR_001	UR_002	UR_003	UR_004	UR_005
FR_001			X	X	
FR_002			X		X
FR_003			X		X
FR_004	X				
FR_005	X				
FR_006			X		
FR_007	X				
FR_008		X			
FR_009			X		
FR_010					X

5. System Behavior

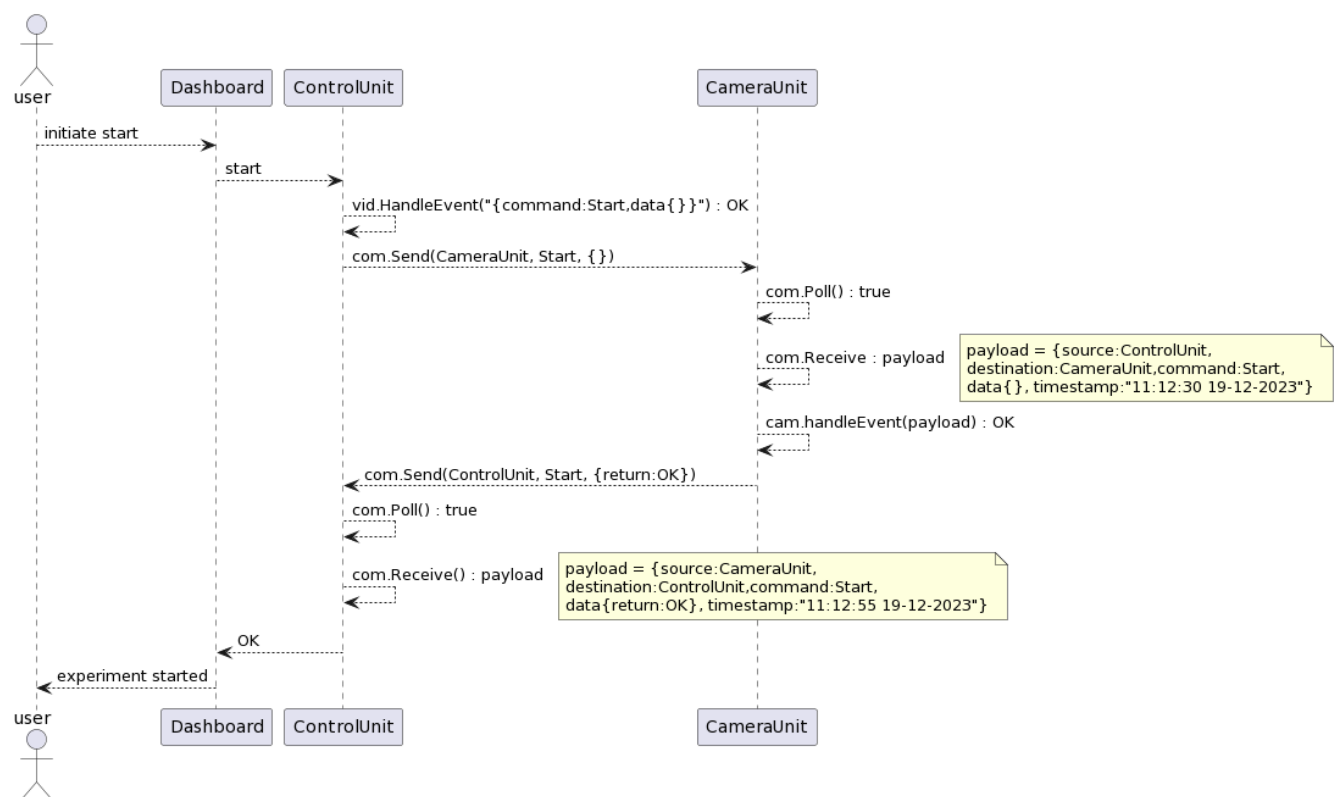
5.1. Sequence Diagrams

5.1.1. UC_001 : Measuring data from a lab setup

As the "Start" and "Stop" command work in the same pattern, only one is shown for the purpose of showcasing the sequence of actions. Besides that, the process is the same for the SensorUnit.

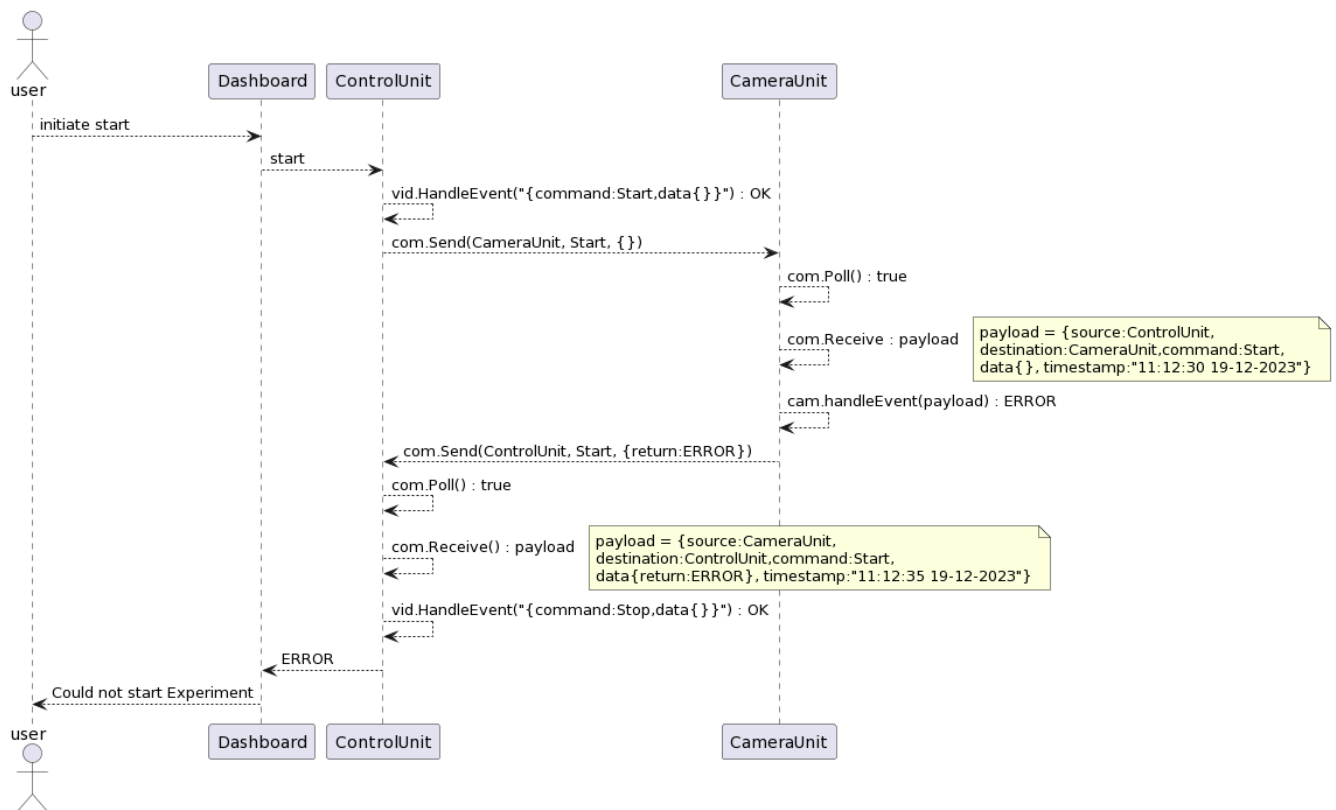
Scenario 1

When the user makes use of the lab setup, the system should be started and measurements should be captured.



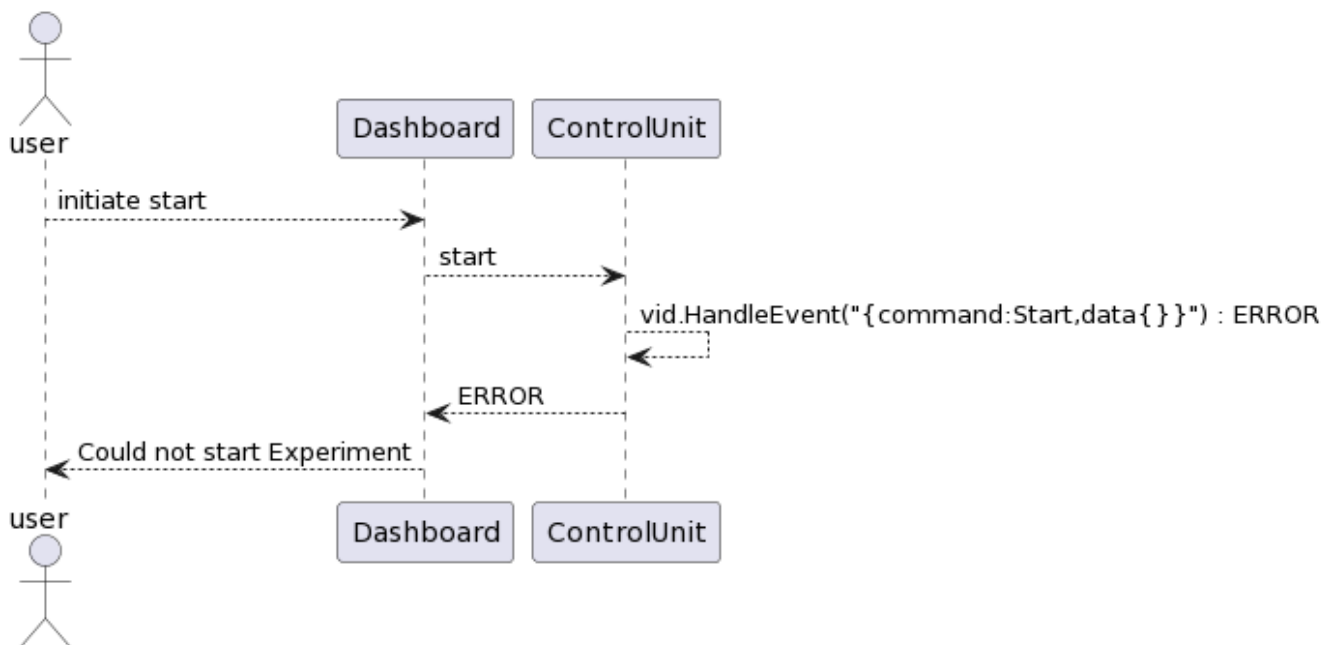
Scenario 2

When the user wants to make use of the lab setup while the system does not have any camera's attached to the CameraUnit, the system should inform the user.



Scenario 3

When the user wants to make use of the lab setup while the system does not have any camera's attached for streaming, the system should inform the user.

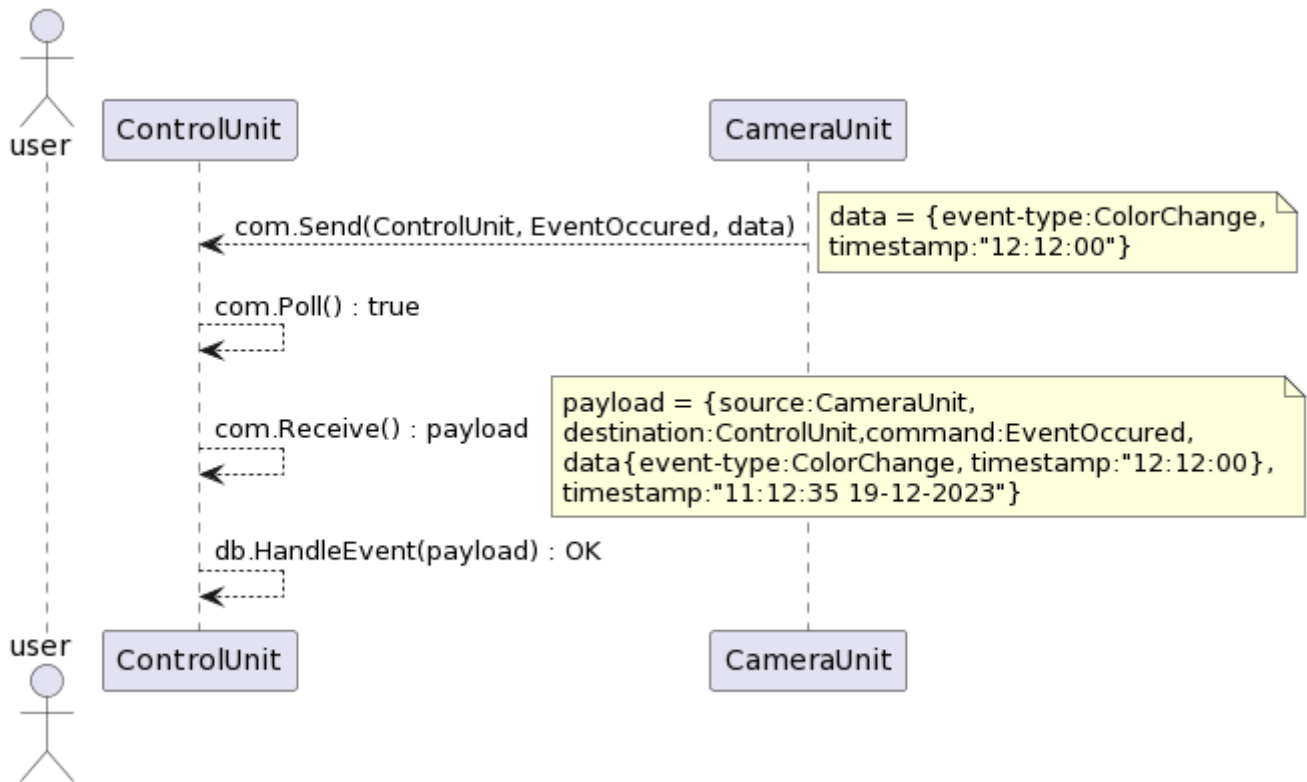


5.1.2. UC_002 : Measured data stored in database

For the purpose of focussing on the particular functionality of this use case, the starting of an experiment is given in the previous use case, [Section 5.1.1](#). The process is the same for data being captured by the camera and by the sensor, the only difference is the unit name, command and data.

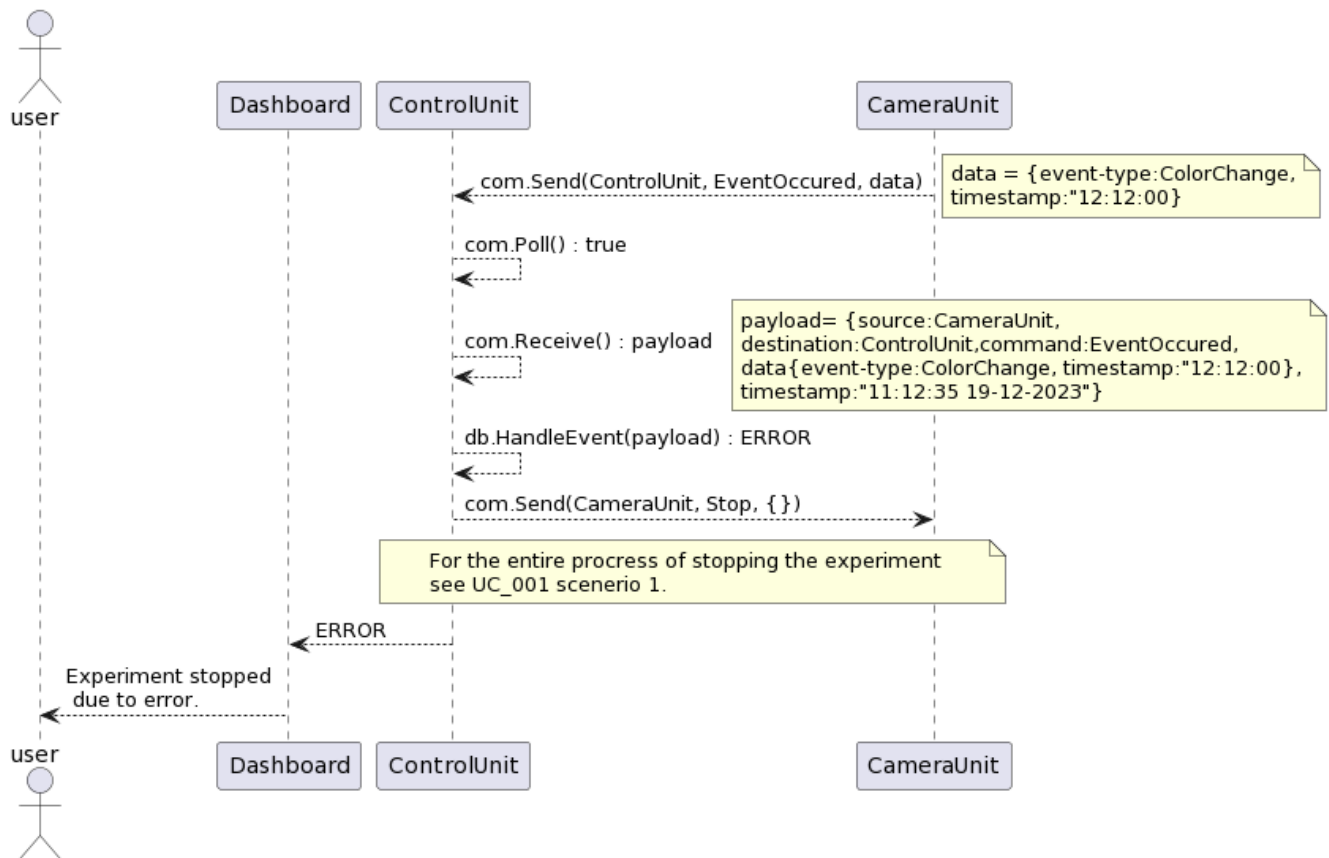
Scenario 1

When an experiment has been started, data must be stored in the database. This is done while the experiment is running.



Scenario 2

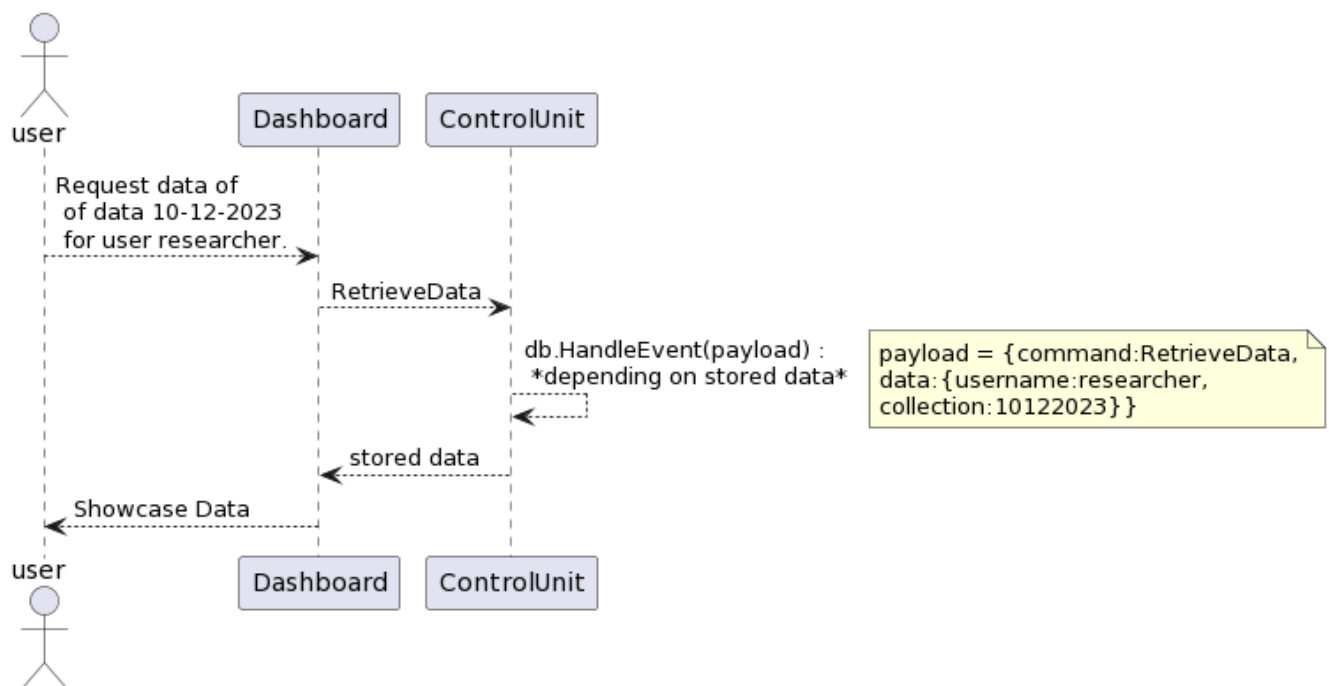
Upon trying to store data in the database, the database has a problem, the experiment must be stopped and the user should be informed.



5.1.3. UC_003 : Retrieving data from database

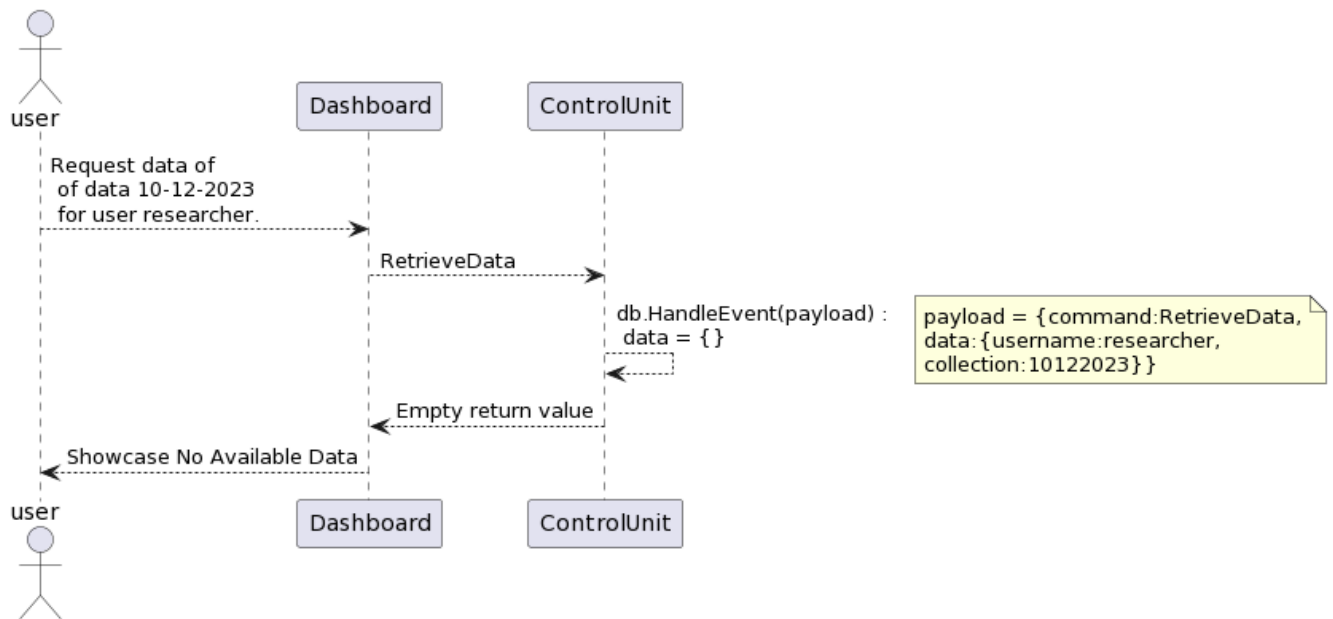
Scenario 1

When the user request for data to be retrieved, the data must be retrieved from the database and showcased on the dashboard.



Scenario 2

When the user request for data to be retrieved, but no data is available and therefor results in an empty value.



5.2. Class Diagram

The Class Diagram is a vital element in our system design, offering a visual representation of classes, their attributes, and the relationships defining our software's static structure. This section provides insight into the entities, attributes, associations, and behaviors that form the foundational architecture of our system.

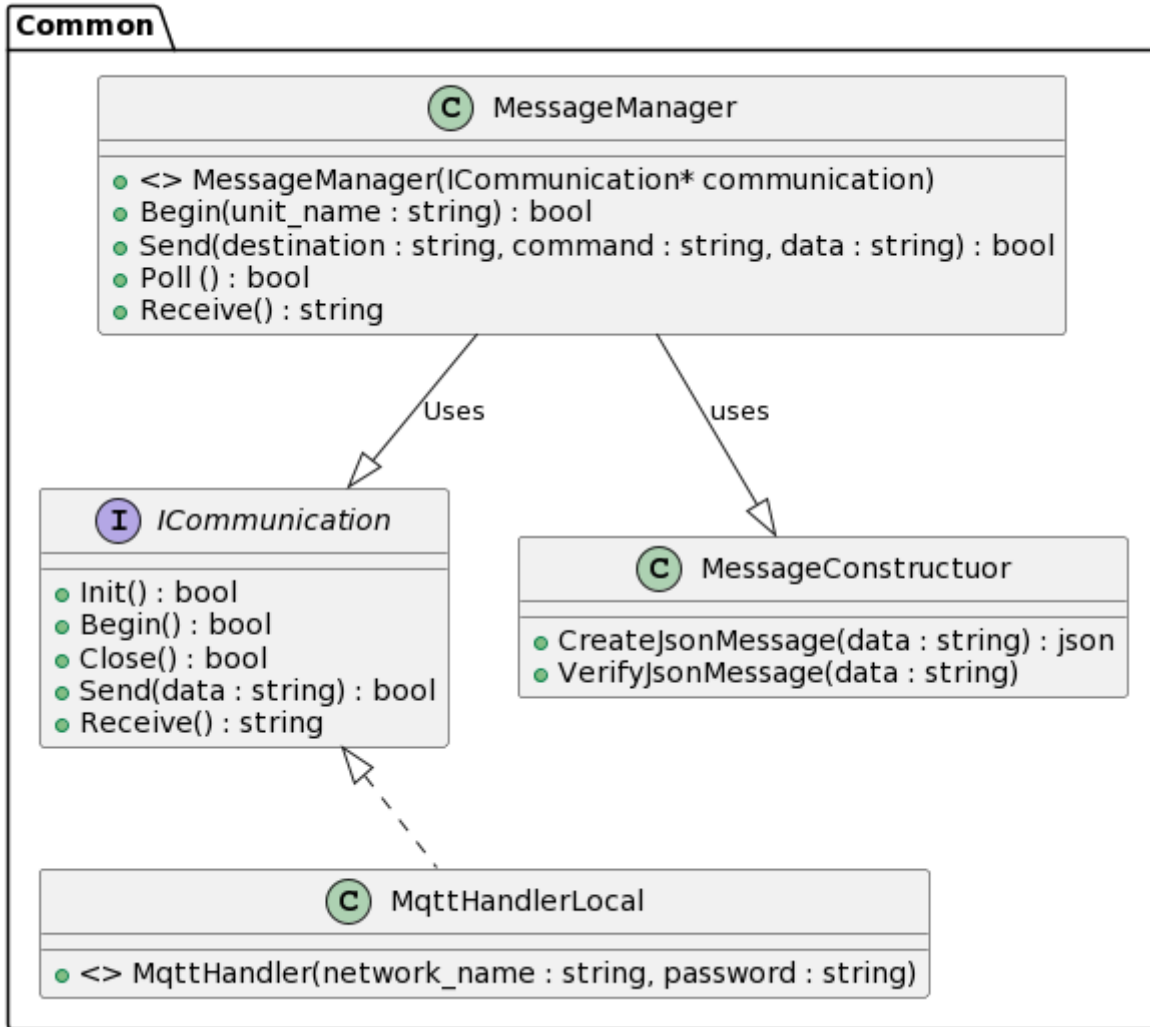


Figure 2. Class Diagram Common

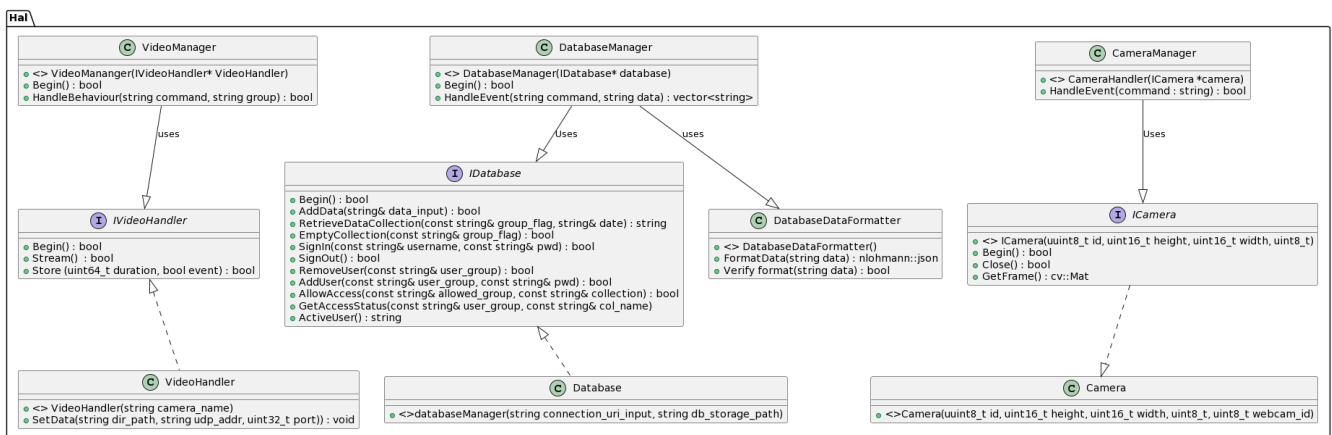


Figure 3. Class Diagram Hal

6. Hardware Diagram

The system will be composed of several hardware modules. Some logical units run on multiple physical blocks. The table, [\[Used hardware components\]](#), describes the individual hardware components. The diagram, [\[Hardware connection\]](#), the connection between the various hardware components.

Table 5. Used hardware components

Hardware block	Description
Raspberry Pi 4 (ControlUnit)	This device is responsible for making any type of decisions and overall generating the musical compositions.
Raspberry Pi 4 (CameraUnit)	This device is responsible for controlling the camera's.
ESP32 (SensorUnit)	This device is responsible for controlling the sensors.
Database	The database is a service which is used for data storage.
Dashboard	The interface is a service running on a computer, to allow for users to interact with the system.
Sensors	Various sensors are used to measure data generated during the experiment.
Camera	The devices that takes care of tracking the expierments progress.

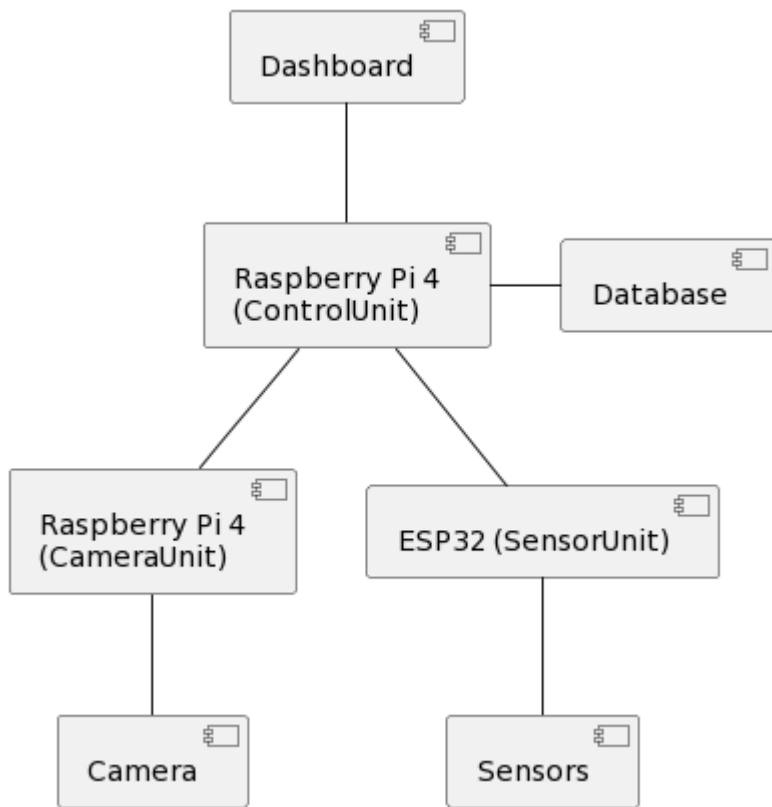


Figure 4. Hardware connection